

:- Object Oriented Programming Using C++ :-

What is OOP?

Object-Oriented Programming (**OOP**) is a powerful paradigm in programming that organizes software design around **objects**, rather than functions and logic.

- An **object** represents a real-world entity. It has:
 - **Attributes (data members / properties)** → Example: name, age, salary.
 - **Behaviors (member functions / methods)** → Example: work(), study(), drive().
- A **class** is a **blueprint** for creating objects.
 - Example: A Car class can be used to create different car objects with unique attributes (like color, model) but sharing the same structure.

In short, OOP allows programmers to structure complex software in a way that is more **modular, maintainable, and reusable**.

Why OOP?

Traditional **Procedure-Oriented Programming (POP)** languages like C focused on **functions** and the flow of the program.

OOP, on the other hand, focuses on **objects**.

Advantages of OOP over POP:

- Encourages **data hiding and security**.
 - Promotes **reusability** of code via inheritance.
 - Provides **flexibility** through polymorphism.
 - Allows **modular programming** – programs can be divided into manageable chunks.
-

POP vs OOP – Comparison

Feature	POP (C)	OOP (C++, Java, Python)
Focus	Functions & procedures	Objects & classes
Data security	Low (data often global)	High (Encapsulation & Data Hiding)
Code Reusability	Difficult, leads to repetition	Achieved through Inheritance
Flexibility	Less flexible	More flexible (Abstraction & Polymorphism)
Example Languages	C, Fortran	C++, Java, Python, C#, Ruby

Features of OOP

1. **Encapsulation** – Wrapping data & functions into a single unit (class).
 2. **Abstraction** – Hiding unnecessary details, showing only essentials.
 3. **Inheritance** – One class acquires properties of another.
 4. **Polymorphism** – One name, many forms (functions/operators).
 5. **Modularity** – Breaking big program into smaller objects.
 6. **Reusability** – Code can be reused with little modification.
-

Abstraction

Definition:

Abstraction is the process of **hiding complex implementation details** and showing only the **necessary features** to the user.

- Users don't need to know **how** something works internally, just **what it does**.
- In C++, abstraction can be achieved using:
 - Abstract classes (classes with at least one pure virtual function).
 - Functions that expose essential functionality only.

Example :

```
#include <iostream>
```

```
using namespace std;
```

```
class Car {
```

```
public:
```

```
    void startEngine() {
```

```
        cout << "Car engine started.\n";
```

```
    }
```

```
};
```

```
int main() {
```

```
    Car c;
```

```
    c.startEngine();
```

```
    // User just calls startEngine()
```

```
    // Internals of how engine works is hidden.
```

```
}
```

- The user only knows about the startEngine() function.
- The **internal engine mechanism** is abstracted away.

Data Hiding

Definition:

Data hiding is a concept where certain details of an object are **restricted** from outside access.

- Achieved using **Access Specifiers**:
 - **private** – Accessible only inside the class.
 - **protected** – Accessible inside the class & its derived classes.
 - **public** – Accessible from anywhere.

Example:

```
class Student {  
private:  
    int marks; // hidden data  
  
public:  
    void setMarks(int m) { marks = m; } // controlled access  
    int getMarks() { return marks; } // controlled access  
};
```

- Direct access to marks is not possible.
- User can only modify/access it via setMarks() and getMarks().

Benefits of Data Hiding:

- Prevents accidental modification.
 - Provides security & control over data.
-

Inheritance

Definition:

Inheritance is a mechanism where one class (derived) **acquires properties & behaviors** of another class (base).

Types of Inheritance in C++:

1. **Single Inheritance** – One base → One derived.
2. **Multiple Inheritance** – One derived → Many bases.
3. **Multilevel Inheritance** – Derived from another derived.
4. **Hierarchical Inheritance** – Many derived from one base.
5. **Hybrid Inheritance** – Combination of above.

Example:

```
class Person {  
public:  
    string name;  
    void setName(string n) { name = n; }  
};  
  
class Student : public Person {  
public:  
    int roll;  
    void show() { cout << name << " " << roll; }  
};
```

- Student class inherits name and setName() from Person.
 - Promotes **reusability** of code.
-

Polymorphism

Definition:

Polymorphism = “One name, many forms.”

It allows functions/operators to behave differently depending on context.

Types of Polymorphism in C++:

1. Compile-Time Polymorphism

- Function Overloading → same function name, different parameters.
- Operator Overloading → operators redefined for user-defined objects.

2. Run-Time Polymorphism

- Achieved using **virtual functions**.
- Base class pointer calls derived class function at runtime.

Example:

```
class Shape {  
public:  
    virtual void draw() { cout << "Drawing Shape\n"; }  
};  
  
class Circle : public Shape {  
public:  
    void draw() override { cout << "Drawing Circle\n"; }  
};
```

- draw() function behaves differently depending on whether object is Shape or Circle.
-

Classes and Objects

- **Class** → Blueprint containing attributes & functions.
- **Object** → Instance of a class created in memory.

Example:

```
class Student {  
    int roll;  
    string name;  
public:  
    void setData(int r, string n) { roll = r; name = n; }  
    void showData() { cout << roll << " " << name; }  
};  
  
int main() {  
    Student s1; // object created  
    s1.setData(1, "Amit");  
    s1.showData();  
}
```

Static Data Members & Functions

- **Static Data Member**
 - Shared by all objects.
 - Only one copy exists.
- **Static Function**
 - Can access only static members.
 - Called using class name.

Example:

```
class Demo {  
    static int count;  
public:  
    Demo() { count++; }  
    static void showCount() { cout << "Count = " << count; }  
};  
int Demo::count = 0;
```

Friend Function

A **friend function** is a function that is **not a member of the class**, but it is given special permission to access private and protected members of that class by declaring it with the keyword **friend**.

1. Declared inside the class using the **friend** keyword, but defined outside the class like a normal function.
2. Not a member function of the class.
3. Can access **private** and **protected** members of the class.
4. Does not require an object of the class to be invoked (called like a normal function).
5. It can be a **standalone function** or a **member of another class**.
6. It is not affected by the access specifiers (private, protected, public) where it is declared.

Syntax

```
class ClassName {  
    private:  
        int data;  
    public:  
        ClassName(int d) { data = d; }  
        friend void showData(ClassName c); // declaration of friend function  
};  
  
// Definition outside the class  
void showData(ClassName c) {  
    cout << "Data = " << c.data; // accessing private member  
}
```

Rules of Friend Functions

1. Friendship is **not inherited**. A derived class does not inherit the friend status of the base class.
2. Friendship is **not mutual**. If class A is a friend of class B, class B does not automatically become a friend of class A.
3. Friendship is **not transitive**. If class A is a friend of B, and B is a friend of C, then A does not automatically become a friend of C.
4. Friend functions can be declared in any access section: private, protected, or public.

Advantages of Friend Functions

1. Provides flexibility when two or more classes need to share data.
 2. Helpful when operator overloading requires access to private data.
 3. Allows external functions to work closely with class members without being actual members.
 4. Useful for debugging and testing private data.
-

Disadvantages of Friend Functions

1. Breaks the concept of **encapsulation** (data hiding) because it allows non-member access to private data.
 2. Too many friend functions can make the program complex and less secure.
 3. Friendship should be used only when absolutely necessary, otherwise it reduces modularity.
-

Constructors

- Special member function with **same name as class**.
- Automatically invoked when an object is created.
- Used to **initialize data members**.

Types:

- Default Constructor
 - Parameterized Constructor
 - Copy Constructor
-

Destructors

- Special function with ~classname.
- Automatically called when object is destroyed.
- Used to release resources.

Example:

```
class Test {  
public:  
    Test() { cout << "Constructor called\n"; }  
    ~Test() { cout << "Destructor called\n"; }  
};
```

Benefits of OOP

- **Data Security** (Encapsulation)
 - **Code Reusability** (Inheritance)
 - **Flexibility** (Polymorphism)
 - **Easier Maintenance**
 - **Real-world Modeling**
-

Applications of OOP

- Software Development (C++, Java, Python)
- Game Development
- Database Systems
- Operating Systems
- Web & Mobile Apps
- Simulation & Robotics